

Let us revisit the neuron pruning. As we know optimising for the mask and the weights is a NP hard problem. So let us assume we have a mask , can we then solve the problem?

Yes!. It accepts a closed form solution!

$$\mathbf{w}_{M_i}^i = (\mathbf{X}_{M_i} \mathbf{X}_{M_i}^\top)^{-1} \mathbf{X}_{M_i} (\mathbf{w}_{M_i} \mathbf{X}_{M_i})^\top,$$

Where w_{M_i} is the mask for row i $\mathbf{w}_{M_i}$ is the smaller weight vector keeping only weights from the mask and similarly $\mathbf{X}_{M_i}$ is the data with dimensions from M_i and $\mathbf{w}_{M_i}^i$ is the updated weight .

If we do this then what is the complexity of the algorithm?

Dominated by the hessian inverse (d^3) So again $O(d_{row} * d^3)$. Note that hessian depends on the data not on the weights. But we are not able to reuse the computation because the masks are different leading to different row and columns from the full hessian and thus the inverses required are different?

What do we do?

i) make the masks same?

Would work for efficiency – lead to structured pruning .. generally known to be bad for accuracy.

ii) so what is the way around it?

we know the following relation between inverses of sub matrices. The main bottleneck is hessian inverse. What we were to use one hessian for another. Does it remind you of some identity ?

In words, starting with U_1 being the set of all indices, each subset U_{j+1} is created by removing the smallest index from the previous subset U_j . These subsets also impose a sequence of inverse Hessians $(\mathbf{H}_{U_j})^{-1} = ((\mathbf{X}\mathbf{X}^\top)_{U_j})^{-1}$ which we are going to share across all rows of $\mathbf{W}$. Crucially, following (Frantar et al., 2022b), the updated inverse $(\mathbf{H}_{U_{j+1}})^{-1}$ can be calculated efficiently by removing the first row and column, corresponding to j in the original $\mathbf{H}$, from $\mathbf{B} = (\mathbf{H}_{U_j})^{-1}$ in $O(d_{col}^2)$ time via one step of Gaussian elimination:

$$(\mathbf{H}_{U_{j+1}})^{-1} = \left(\mathbf{B} - \frac{1}{[\mathbf{B}]_{11}} \cdot \mathbf{B}_{:,1} \mathbf{B}_{1,:} \right)_{2:,2:}, \quad (5)$$

Can we use it somehow. ? To conform to the identity the subsequent Hessians we would have to build would have to be one rank less. Lets say we compute hessian inverse for full matrix assuming there is no mask, what matrix inverse is easy to compute from that reusing the matrix?

Any matrix with one column removed (one channel /dimension removed). So maybe that is what we use. So lets do a sweep!!!

We compute H^{-1} – mask the first column $\Rightarrow$ get find the solution for rest of the weights using H^{-1} then find $H_{:,1:}$, mask the next column as per the mask, use this hessian to update the weights on the right.

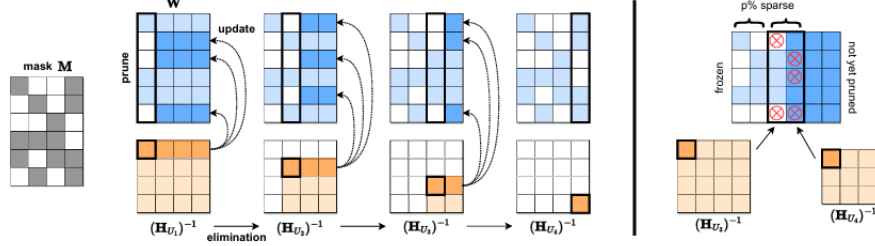


Figure 4. [Left] Visualization of the SparseGPT reconstruction algorithm. Given a fixed pruning mask M , we incrementally prune weights in each column of the weight matrix W , using a sequence of Hessian inverses $(H_{U_j})^{-1}$, and updating the remainder of the weights in those rows, located to the “right” of the column being processed. Specifically, the weights to the “right” of a pruned weight (dark blue) will be updated to compensate for the pruning error, whereas the unpruned weights do not generate updates (light blue). [Right] Illustration of the adaptive mask selection via iterative blocking.

Weight Freezing Interpretation. While we have motivated the SparseGPT algorithm as an approximation to the exact reconstruction using optimal partial updates, there is also another interesting view of this scheme. Specifically, consider an exact greedy framework which compresses a weight matrix column by column, always optimally updating all not yet compressed weights in each step (Frantar et al., 2022b;a). At first glance, SparseGPT does not seem to fit into this framework as we only compress some of the weights in each column and also only update a subset of the uncompressed weights. Yet, mechanically, “compressing” a weight ultimately means fixing it to some specific

value and ensuring that it is never “decompressed” again via some future update, i.e. that it is *frozen*. Hence, by defining column-wise compression as:

$$\text{compress}(w^j)_i = 0 \text{ if } j \notin M_i \text{ and } w_i^j \text{ otherwise,} \quad (6)$$

i.e. zeroing weights not in the mask and fixing the rest to their current value, our algorithm can be interpreted as an exact column-wise greedy scheme. This perspective will allow us to cleanly merge sparsification and quantization into a single compression pass.

So far, we have focused only on weight reconstruction, i.e. assuming a fixed pruning mask $\mathbf{M}$. One simple option for deciding the mask, following AdaPrune (Hubara et al., 2021a), would be via magnitude pruning (Zhu & Gupta, 2017). However, recent work (Frantar et al., 2022b) shows that updates during pruning change weights significantly due to correlations, and that taking this into account in the mask selection yields better results. This insight can be integrated into SparseGPT by *adaptively* choosing the mask while running the reconstruction.

One obvious way of doing so would be picking the $p\%$ easiest weights to prune in each column i when it is compressed, leading to $p\%$ overall sparsity. The big disadvantage of this approach is that sparsity cannot be distributed non-uniformly across columns, imposing additional unnecessary structure. This is particularly problematic for massive language models, which have a small number of highly-sensitive outlier features (Dettmers et al., 2022; Xiao et al., 2022).

We remove this disadvantage via *iterative blocking*. More precisely, we always select the pruning mask for $B_s = 128$ columns at a time (see Appendix A), based on the OBS reconstruction error ε from Equation (3), using the diagonal values in our Hessian sequence. We then perform the next B_s weight updates, before selecting the mask for the next block, and so on. This procedure allows *non-uniform selection* per column, in particular also using the corresponding Hessian information, while at the same time considering also previous weight updates for selection. (For a single column j , the selection criterion becomes the magnitude, as $[\mathbf{H}^{-1}]_{jj}$ is constant across rows.)

This can also be applied to structured sparsity by applying it on groups of OBS on groups of M weights and pruning N weights.